

1. What does the loss function measure during training?

1 point

- ☐ The rate at which the model is learning
- ☐ The accuracy of the model's predictions.
- ☐ How confident the model is in its predictions
- ☒ The error between predictions and actual values.

2. Why do neural networks need activation functions like ReLU between layers?

1 point

- ☐ To reduce the amount of data needed for training.
- ☐ To speed up the training process.
- ☐ To automatically adjust the learning rate during training.
- ☒ To enable the network to learn non-linear patterns.

3. If your training process runs for 1000 epochs, what does one epoch represent?

1 point

- ☐ One update to all the model's weights
- ☐ One forward and backward pass through the network
- ☐ One complete pass through the entire training dataset.
- ☒ One pass through a single batch of training data.

4. In PyTorch, what does `loss.backward()` accomplish?

1 point

- ☒ Calculates gradients to reduce the loss.
- ☐ Saves the trained model parameters to disk.
- ☐ Cleans and prepares data before training.
- ☐ Makes predictions using the neural network.

5. What does the `dtype` parameter control when creating a PyTorch tensor?

1 point

- ☐ The number of dimensions in the tensor.
- ☐ How many samples are in the batch.
- ☐ The device where the tensor is stored (CPU or GPU).
- ☒ The type of numbers stored in the tensor.

6. What will be printed by the following code?

1 point

```
1 x = torch.tensor([2.0, 4.0, 6.0])
2 y = torch.tensor([1.0, 2.0, 3.0])
3 result = x / y + 5
4 print(result)
```

- ☒ 1 tensor([7., 7., 7.])

- ☐ 1 tensor([0.3333, 0.5714, 0.7500])

- ☐ 1 tensor([7., 4.5, 3.6667])

- ☐ 1 tensor([2., 2., 2.])

7. How does PyTorch apply mathematical operations to tensors?

1 point

- ☒ To each element in the tensor.
- ☐ Only to tensors with exactly the same shape.
- ☐ By converting tensors to Python lists first.
- ☐ Only to the first element in the tensor.

8. What is one key advantage of using PyTorch tensors instead of Python lists for neural network training?

1 point

- ☐ Tensors can store more data than Python lists.
- ☐ Tensors eliminate the need for data preprocessing.
- ☒ Tensors are optimized for mathematical operations.
- ☐ Tensors automatically adjust the learning rate during training.

9. What will be printed by the following code?

1 point

```
1 x = torch.tensor([25.0])
2 x = x.unsqueeze(0)
3 x = x.squeeze()
4 print(x.shape)
```

- ☐ 1 torch.Size([1, 1])

- ☐ 1 torch.Size([2])

- ☐ 1 torch.Size([])

1 torch.Size([1])

10. In PyTorch, what is "broadcasting"?

1 point

- ☒ Expanding tensors to compatible shapes for operations.
- ☐ Compressing tensors to save GPU memory.
- ☐ Converting tensor data types during operations.
- ☐ Reducing the dimensions of tensors to simplify computations.